Two Monolingual Transformer Language Models from Scratch: Hindi (Model H) and Nepali (Model L)

Final Project Report: Phases 1–3 and Bonus Ablation

Yogendra Patel Roll No. 2025201098 CL3410 Language Models and Agents, Monsoon 2026, IIIT Hyderabad

Code: github.com/CL3-410/individual-project-CReativeRebe1 Checkpoints and data: Google Drive Dataset: HuggingFace

Summary

I built two separate decoder-only language models of 23.85M parameters each. Model H is trained on Hindi and Model L on Nepali. Each model has its own corpus, tokenizer, weights and finetuning data. Both languages use Devanagari, so language identification had to rely on vocabulary. Table 1 lists the main numbers from every phase.

Table 1: Main results of the project. EM = exact match on the synthetic reasoning test sets (SFT + replay, 3 seeds).

	Model H	**Model L**		**Model H**	**Model L**
Corpus tokens (all splits)	517.6M	572.5M	Test perplexity	31.01	52.16
Manually collected share	22.5%	24.7%	Test bits-per-byte	0.4819	**0.4561**
Tokenizer	Unigram 16k	Unigram 16k	Pretrained EM (all splits)	0.000	0.000
Fertility (tokens/word)	1.273	1.380	EM, seen names	**0.422**	0.368
Parameters	23,846,784	23,846,784	EM, unseen names + unseen wording	0.102	0.107
Tokens per parameter	21.6	24.0	Val PPL after SFT (with replay)	142 (35.5)	418,137 (76.3)

1 Phase 1: Data Collection and Tokenizers

1.1 Languages and sources

- **Hindi** is the higher-resource language and **Nepali** is the lower-resource one (it is on the allowed list). Nepali Wikipedia has 8.5M words and Hindi has 51.9M.
- Choosing two languages in the same script makes the setup harder. A Hindi and a Nepali page cannot be separated by character set, and one normalisation rule turned out to be right for Hindi and wrong for Nepali (below).
- Manual side: my own crawler, a fast news crawler and Wikipedia dumps. Downloaded side: IndicCorpV2. Duplicates are removed only from the downloaded side, so the manual share is not lost.

Table 2: Raw collection, before cleaning (words).

	Crawler	Fast crawler	Wikipedia	**Manual total**	IndicCorpV2
Hindi	27.7M (68k docs)	79.0M (103k)	51.9M (117k)	**158.6M (288k docs)**	600.0M (6.40M docs)
Nepali	17.2M (28k docs)	141.7M (313k)	8.5M (31k)	**167.3M (371k docs)**	600.0M (10.12M docs)

1.2 Cleaning pipeline

The manual text goes through the stages below in this order. Each stage streams the data, so the full corpus never has to fit in the 7.4 GB of RAM on my laptop.

1. **Language filter:** Devanagari ratio, then fastText `lid.176` confidence, then a Hindi-vs-Nepali marker-word vote.
2. **Rule-based repair** (moved before filtering): removes JSON blobs, boilerplate lines, repeated punctuation and mojibake. If a repair would delete more than 50% of a document's characters, the document is kept unchanged.
3. **Text-metric rules** (11 Gopher/C4-style rules made Devanagari-aware) and **repetition rules** (top n-gram share, duplicate 5-grams).
4. **Perplexity** (character 4-gram LM trained on Wikipedia), **quality classifier**, and **coherence** (TF-IDF cosine between paragraphs over character 4-grams).
5. **Deduplication:** cross-document paragraphs first, then exact SHA-256, then MinHash-LSH (128 permutations, $J \geq 0.8$, `xxhash` so results are the same across processes).
6. **Topic, safety and PII:** TF-IDF + k-means with $k = 60$ (ad-heavy clusters cut to 25%), blocklist, PII redaction.
7. **Decontamination:** 13-gram overlap between training and held-out text.

Table 3: Thresholds that differ by language, and what happens to the manual documents.

Setting	Hindi	Nepali	Funnel (manual documents)	Hindi	Nepali
fastText min. confidence	0.65	0.50	Collected	288,469	370,801
Quality classifier min.	0.30	0.10	After language filter	93.3%	93.0%
Perplexity cutoff (p99 of wiki)	16.68	18.01	After repair + rules	68.3%	82.7%
Coherence min. (p1 of wiki)	0.0123	0.0106	After perplexity/quality/coherence	54.5%	77.2%
Chandrabindu folded to anusvara	yes	**no**	After all dedup + topic + safety	**52.9%**	**73.2%**

- Thresholds come from measurements. With the Hindi confidence cutoff of 0.65, fastText rejects 17.9% of genuine Nepali Wikipedia; at 0.50 it rejects 3.0%. A quality cutoff of 0.30 would keep only 19.2% of the Nepali crawl.
- Normalisation: Hindi folds chandrabindu into anusvara. Nepali must keep it (*gāuñ*, *bāñkī*), because folding produces misspelt Nepali.
- Language ID on 3,000 Wikipedia articles per language: fastText is 99.97% correct on Hindi and 95.87% on Nepali. The errors go in one direction only: 119 Nepali articles were labelled Hindi and none the other way. The marker vote gets 99.97% on Nepali, so both checks are used.
- Deduplication removed 510k paragraphs, 378 exact and 1,343 near duplicates in Hindi, and 1.84M paragraphs, 1,534 exact and 6,652 near duplicates in Nepali.
- Decontamination was applied to the held-out side. Dropping contaminated *training* documents would have removed 46.3% of the Nepali manual documents and pushed the manual share down to 15%, below the 20% floor. Instead, held-out candidates that collide with training text are replaced. Hindi discarded 16.5% of candidates and Nepali 43.0%, because Nepali news sites reuse agency sentences heavily.

1.3 Final corpus and splits

Table 4: Final corpora. The 20% manual floor is checked in code, and the pipeline fails if the share falls below it.

	Words	Tokens	Manual share (tokens)	Train / val / test docs	Train tokens	Val / test tokens
Hindi	409.0M	517,640,653	22.5%	3,577,045 / 5,000 / 5,000	515,972,223	821,762 / 846,668
Nepali	414.4M	572,506,257	24.7%	5,526,928 / 5,000 / 5,000	571,227,702	634,788 / 643,767

1.4 Tokenizer

- Sweep: SentencePiece BPE and Unigram at 6 vocabulary sizes, giving 12 tokenizers per language. Each was trained on a stride sample of its own training split and scored on the real validation split. Settings: `character_coverage=0.9995`, `byte_fallback=True`.
- A bug I hit: SentencePiece silently drops lines longer than 4,192 bytes, which is about 1,400 Devanagari characters. The training input is now split on the danda.
- The UNK rate is 0.0% for all 24 tokenizers because byte fallback is on, so fertility decided the choice. Unigram beats BPE at every vocabulary size in both languages.
- **Chosen: Unigram, 16k, for both languages.** At 48k the embedding table alone would use the whole 25M budget. At 16k it costs 33% of the budget when $d = 512$ (26% at the final $d = 384$). The price is about 3% worse fertility than 24k, and 16k also uses more of its vocabulary.

Table 5: Held-out tokenizer metrics (fertility = tokens per word; utilisation = share of the vocabulary used on validation text).

	Hindi				Nepali				Embedding share
Vocab	BPE fert.	Uni. fert.	Uni. chars/tok	Uni. util.	BPE fert.	Uni. fert.	Uni. chars/tok	Uni. util.	of 25M (d=512)
8k	1.420	1.389	3.742	0.977	1.581	1.541	4.222	0.972	16%
12k	1.339	1.314	3.955	0.978	1.466	1.439	4.521	0.975	25%
16k	1.293	**1.273**	**4.084**	**0.970**	1.402	**1.380**	**4.714**	**0.970**	33%
24k	1.242	1.231	4.224	0.915	1.327	1.315	4.950	0.931	49%
32k	1.216	1.207	4.305	0.841	1.285	1.277	5.097	0.877	66%
48k	1.188	1.184	4.389	0.695	1.239	1.234	5.271	0.762	98%

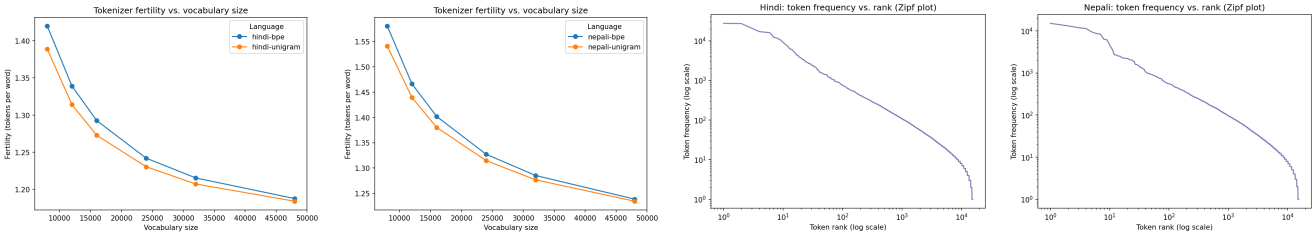


Figure 1: Fertility sweep (left two) and token frequency against rank on held-out text (right two). Both follow a Zipf curve.

Tokenization examples. I segmented four held-out sentences per language with both tokenizers. Using the other language's tokenizer costs Hindi text **+79%** tokens (43 to 77) and Nepali text **+70%** (43 to 73). The Nepali name *Prachaṇḍale* is one piece under the Nepali tokenizer and four under the Hindi one. Hindi *baṛā* contains a nukta, which Nepali rarely uses, so the Nepali tokenizer falls back to three raw UTF-8 bytes for it. Suffixes such as Nepali *-harū*, *-le*, *-mā* are single pieces in their own tokenizer and get split into characters by the other. The two vocabularies share 5,695 of their 16,000 pieces, mostly common Sanskrit-derived substrings.

2 Phase 2: Model, Pretraining and Evaluation

2.1 Architecture

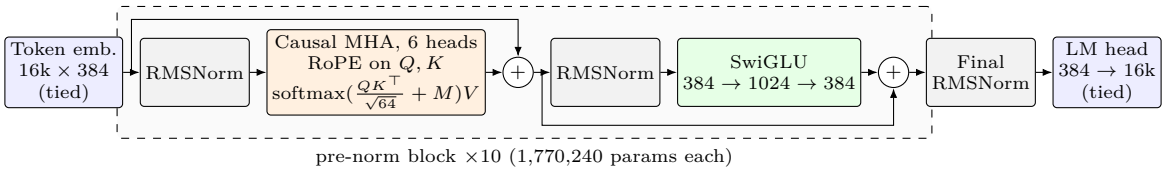


Figure 2: Forward pass. Model H and Model L use the same code but have separate weights, vocabularies and data.

Table 6: Configuration and parameter count (both models are identical in shape).

Hyperparameter	Value	Component	Parameters
d_{model} / layers / heads / d_k	384 / 10 / 6 / 64	Token embedding (tied with head)	6,144,000
FFN (SwiGLU) hidden	1,024 = (8/3)d	Attention per block ($4d^2$)	589,824
Context length	1,024	SwiGLU per block ($3 \cdot d \cdot 1024$)	1,179,648
Norm / position	pre-norm RMSNorm / RoPE	RMSNorm gains per block	768
Dropout / weight tying	0.0 / yes	10 blocks + final norm	17,702,784
Init	$\mathcal{N}(0, 0.02^2)$, residual writers $\times 1/\sqrt{2N}$	**Total**	**23,846,784**

- Depth vs width: The embedding costs 16,000 × d whatever the depth. At $d = 512$ only about 6 layers fit in the budget; at $d = 384$ ten fit. The config computes the parameter count at import and raises an error outside 23–25M. An FFN width of $4d$ with SwiGLU would have given 29.7M.
- RoPE instead of a learned table. A learned table would cost 1024 × 384 = 393,216 parameters, while RoPE has none. With RoPE, $q_m \cdot k_n$ depends only on the distance $m - n$ (a test checks this). There is no position table to run out of, so the length limit is soft and is set by the cos/sin cache, which is rebuilt from the config. Quality still drops beyond the 1,024 tokens seen in training.
- Attention shapes: $[B, T, d] \to$ fused $QKV \to$ `view` $[B, T, 6, 64] \to$ `transpose` $[B, 6, T, 64]$, and back through `contiguous().view`. Scores are divided by $\sqrt{d_k}$: the dot product of two 64-dimensional vectors has a standard deviation of about 8, which would make the softmax nearly one-hot with vanishing gradients.
- The causal mask is checked by behaviour: changing token t must leave every logit before t unchanged (within 10^{-6}) and must change the logit at t. Training uses the fused SDPA kernel; the analysis uses an explicit path that returns A. A test checks that the two agree within 10^{-5}.
- Pre-norm keeps an un-normalised residual path, so gradients reach layer 0 without passing through ten norms (warmup was only 2%). RMSNorm runs in fp32, because squaring activations overflows fp16. Tying saves 6.1M parameters. Dropout is 0, because every token is seen once in a single epoch.

2.2 Pretraining

- **Setup:** AdamW $\beta = (0.9, 0.95)$ with weight decay 0.1 on matrices only, peak LR 6×10^{-4}, 2% linear warmup, then cosine decay to 0.1× peak, and gradient clipping at 1.0. Batch: 16 sequences × 1,024 tokens × 4 accumulation steps × 2 GPUs = 131,072 tokens/step. Kaggle 2×T4 with DDP and fp16 (T4 cards cannot use bf16). Gradients are unscaled before clipping.
- One epoch over a shuffled permutation of windows, so coverage is exact. Sampling with replacement would cover only about 63% of windows. Each DDP rank takes window indices $w \equiv r \pmod 2$, because the splits are

stored manual-first.

- Checkpoints hold weights, optimizer, scaler, schedule parameters, step, sampler position and RNG states. They are written to a `.tmp` file and then `os.replaced`, so a crash cannot corrupt the last good file. Logs are append-only and tagged with a `run_id`. Model L's Kaggle session died partway through and resumed from `ckpt_last.pt`.

Table 7: Pretraining runs.

	Model H	Model L		Model H	Model L
Steps / warmup	3,936 / 78	4,358 / 87	First / final train loss	9.109 / 3.437	9.271 / 3.826
Tokens seen (coverage)	515.9M (99.99%)	571.2M (100.00%)	Final val loss (PPL)	3.4237 (30.68)	3.9371 (51.27)
Sessions / wall clock	1 / 1h55m	2 / ∼2h00m	Val evals, all improved	16	18
Throughput / GPU memory	75.7k tok/s / 6.72 GB	79.7k tok/s / 6.72 GB	Grad norm mean / clipped steps	0.44 / 4 of 394	0.45 / 5 of 441

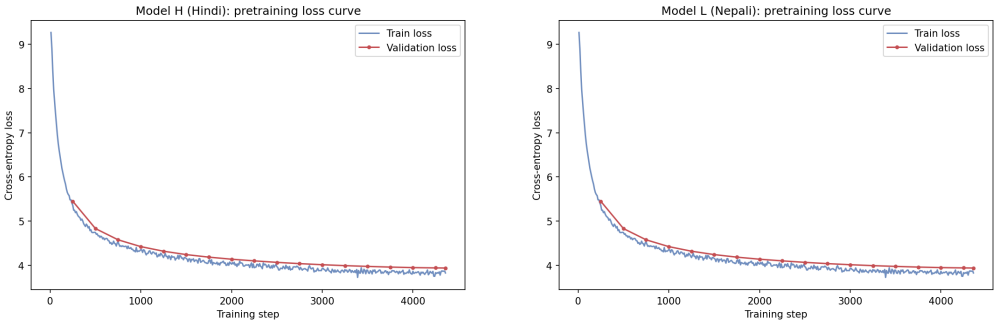


Figure 3: Training and validation loss. Validation loss fell at every evaluation in both runs, so neither model overfitted.

Neither model overfitted. For Model L, the final training loss is even 0.11 above the validation loss. There were no fp16 loss-scale backoffs, and the scale grew in both runs. Both runs are close to Chinchilla-optimal (about 20 tokens/parameter): H 21.6, L 24.0.

2.3 Perplexity and bits-per-byte

Table 8: Language modelling on held-out text. BPB = total NLL in bits divided by the measured UTF-8 bytes.

	H val	**H test**	L val	**L test**
Cross-entropy (nats/token)	3.4124	**3.4344**	3.9364	**3.9544**
Perplexity	30.34	**31.01**	51.23	**52.16**
Bits/token	4.923	4.955	5.679	5.705
Bytes/token	10.376	10.282	12.495	12.507
Bits-per-byte	0.4744	**0.4819**	0.4545	**0.4561**

- Perplexity makes Model H look 40% better, but **per byte Model L is better** (0.4561 vs 0.4819). A Nepali token carries 22% more text. The bits/token ratio (1.151) is smaller than the bytes/token ratio (1.216), so the whole perplexity gap comes from the tokenizers.
- BPB is the only fair metric for comparing the two models. The byte counts are measured by decoding the scored tokens. Test is only 1.8–2.2% worse than validation, and `ckpt_best` is the same checkpoint as `ckpt_last`.

2.4 Generation quality and diversity

Protocol: 200 held-out documents. The first 128 tokens are the prompt, 128 tokens are generated and scored against the true next 128. BLEU is corpus-level BLEU-4. The four settings the assignment names are pure temperature sweeps; the top-k and top-p rows are extra.

Table 9: Generation metrics on the test split (Rep-4 = fraction of repeated 4-grams).

	Model H						Model L					
Setting	chrF	BLEU-4	ROUGE-L	Dist-1	Dist-2	Rep-4	chrF	BLEU-4	ROUGE-L	Dist-1	Dist-2	Rep-4
Greedy	16.60	1.47	11.82	0.062	0.152	0.732	18.35	0.78	7.50	0.082	0.160	0.675
$T = 0.5$	21.34	**1.86**	13.12	0.137	0.453	0.194	23.56	**1.09**	8.56	0.219	0.568	0.152
$T = 1.0$	22.96	0.87	11.38	0.330	0.833	0.002	26.26	0.70	7.33	0.442	0.917	0.001
$T = 1.5$	20.03	0.15	4.59	0.594	0.994	0.000	23.86	0.00	3.03	0.591	0.993	0.000
$T = 0.7$, top-k 50	23.48	1.62	**13.32**	0.170	0.581	0.048	25.70	0.95	**9.06**	0.277	0.722	0.031
$T = 0.8$, top-p 0.9	**23.75**	1.76	13.23	0.208	0.662	0.021	**26.54**	1.17	8.80	0.317	0.789	0.018
$T = 1.0$, top-p 0.95	23.00	1.10	11.80	0.313	0.817	0.001	26.54	0.73	7.57	0.415	0.901	0.002

- Among the pure temperatures, **BLEU and ROUGE-L peak at $T = 0.5$ and chrF peaks at $T = 1.0$**, in

both models. At $T = 1.5$ Dist-2 reaches 0.99 and BLEU falls to about 0.

- BLEU tells us very little here. Open-ended text has no single right continuation, so a fluent but different paragraph scores about the same as gibberish. ROUGE-L uses the longest common subsequence and degrades more gently. chrF is the most useful: both languages are inflected, and a wrong suffix costs chrF a few characters but ruins a whole word n-gram. None of the three can rank H against L.
- Diversity: greedy decoding repeats about 70% of its 4-grams. Self-BLEU is about 0 for the sampled settings and below 0.55 for greedy, so different prompts do not produce the same text. A 30-step checkpoint used as a control got Dist-2 of 0.001 and self-BLEU of 90.9.
- Qualitative ($T = 0.5$): grammar is correct in both languages (Hindi postpositions and agreement; Nepali ergative *-le* and verb morphology). Each sample stays on topic, but most fall into loops after 2–3 sentences. The Nepali output keeps chandrabindu (*batāwñchhan*), which confirms the Phase-1 normalisation choice. Share of Devanagari characters: H 99.71%, L 99.60% (the Nepali reference is 100%). Unique-word ratio against the reference: H 25% vs 47%, L 36% vs 68%.

2.5 Attention analysis

Statistics come from the explicit attention path on 8 validation documents of 192 tokens each. Entropy is given in bits and also normalised by $\log_2(i+1)$, the maximum possible under causal masking. Distance is $\sum_j (i-j)A_{ij}$; prev = attention mass on token $i-1$.

Table 10: Attention by layer (means over 6 heads).

	Model H				Model L			
Layer	bits	norm.	distance	prev	bits	norm.	distance	prev
0	5.610	0.903	43.53	0.033	5.422	0.873	41.11	0.048
2	5.220	0.840	35.66	0.079	5.410	0.872	36.36	0.063
4	5.314	0.858	33.58	0.068	5.077	0.822	31.74	0.093
5	4.863	0.787	29.48	0.087	5.107	0.825	32.01	0.089
6	**4.061**	**0.665**	**17.46**	0.152	**4.006**	**0.654**	**17.43**	0.134
7	4.147	0.673	25.45	0.137	3.680	0.603	17.30	**0.236**
8	3.992	0.650	24.37	**0.156**	3.768	0.604	36.81	0.048
9	3.627	0.588	34.56	0.099	4.070	0.660	30.23	0.113

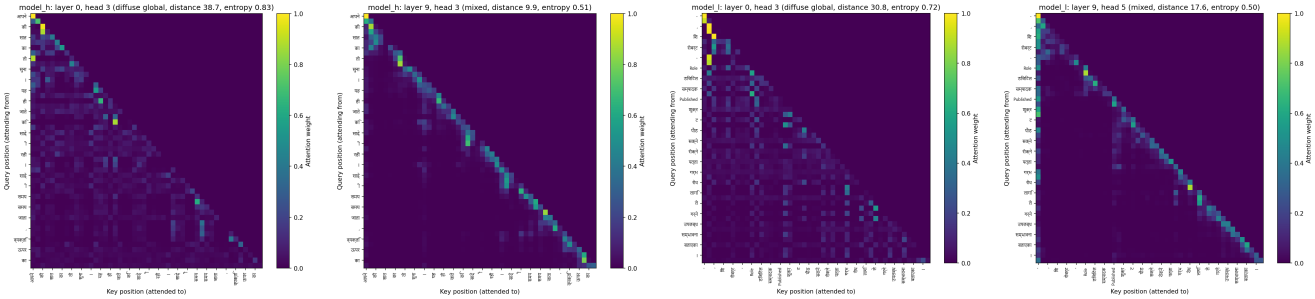


Figure 4: Heatmaps on held-out sentences: Model H layer 0 head 3 and layer 9 head 3, then Model L layer 0 head 3 and layer 9 head 5. Early heads are diffuse; late heads show a band just below the diagonal (recency).

- Attention specialises with depth. Layer 0 is nearly uniform, and by the top layers normalised entropy drops to about 0.6 while previous-token mass is several times higher.
- **Both models focus most at layer 6, at almost the same distance** (17.46 vs 17.43 tokens), even though they share no data or weights. This looks like a property of the architecture and budget, not of the language.
- Head roles (thresholds: prev-token if prev > 0.5; diffuse if norm. > 0.7; long-range if distance > 20): H has 46 diffuse, 9 mixed, 4 long-range, 1 previous-token; L has 42, 10, 7, 1. Each model has exactly one clear previous-token head. The low-entropy heads that attend far back in layers 6–9 are the likely content-based heads.
- Differences: Model L specialises earlier (prev-token mass 0.236 at layer 7) and keeps more long-range heads. One possible reason is its coarser tokens, since 192 tokens cover more text.

3 Phase 3: Reasoning Finetuning and Attention After Finetuning

3.1 Synthetic reasoning dataset

- Every example starts as structured facts (entities, values or relations, and a question), which are then written out by per-language templates. An independent solver recomputes each label by transitive closure, and a test checks that it agrees with every rendered answer.
- Tasks: `cmp2`/`cmp3` compare 2 or 3 explicit values (the answer is an entity, or *barābar* "equal" on a tie); `ext3` asks

for the largest or smallest given $A > B$, $B > C$; `pair3` asks about A vs C, the transitive step itself; `ext4` is a 3-relation chain. Attributes: age, height, weight, price, object weight, quantity.

- Four wording families per language, written by hand. F1 (adjective): *Rāhul, Amit se baṛā hai / Hari Shyāmbhandā jeṭho cha.* F4 (held out): *Amit ke mukābale Rāhul baṛā hai / Shyāmbhandā Hari jeṭho cha.* Each premise is stated in a random direction ($A > B$ or $B < A$), so "pick the first name" does not work.
- Per-language choices, checked against the corpora: Hindi uses ASCII digits (92% of digits in its corpus) and Nepali uses Devanagari digits (90%). Hindi uses a separate postposition (*se*), Nepali an attached suffix (*-bhandā*). Nepali *chan* is written without the final virama, because the Nepali normaliser strips it.
- Leakage control: 200 people and 60 objects per language, split 70/30 into seen and unseen pools. No name is shared across languages and no name is a prefix of another.

Table 11: Splits (identical design in both languages) and measured dataset properties.

	seen wording (F1–F3)	held-out wording (F4)
seen names (140 + 42)	`test_id`	`test_wording`
unseen names (60 + 18)	`test_entity`	`test_both`

Heuristic accuracy on tests (chance 0.30):
first-named 0.24–0.27, last-named 0.26–0.28, most-frequent 0.12–0.13

Property	Hindi	Nepali
Train / val / each test	20k / 1k / 1k	20k / 1k / 1k
Prompt tokens median / max	33 / 56	32 / 49
Observed templates F1/F2/F3/F4	50/43/39/51	53/42/39/54
Prompt / fact overlap across splits	0 / 0	0 / 0
Tokenizer round-trip failures	0	0

3.2 Finetuning setup

- Format: `[EOS]` *praśn:* `<premises> <question>` *uttar:* `<answer> [EOS]`, all on one line, since neither tokenizer round-trips a newline. The loss covers only the answer and the final EOS (labels are −100 elsewhere). The vocabulary stays unchanged.
- Hyperparameters: AdamW $\beta = (0.9, 0.95)$, weight decay 0.1, clipping 1.0, 5% warmup then cosine to 10%, batch 64, 3 epochs (938 steps), dropout 0.1. Model selection uses validation EM every 100 steps (patience 4). Hardware: RTX 3050 Laptop GPU with bf16, 2.5–4 minutes per run. Three seeds per condition.
- Replay condition: each step also includes 8 pretraining windows of 256 tokens (loss weight 1), to limit forgetting.
- LR sweep on val EM: H 0.368 / **0.447** / 0.362 and L 0.339 / 0.356 / **0.370** for 5×10^{-5} / 10^{-4} / 3×10^{-4}. The chosen LRs are H 10^{-4} and L 3×10^{-4}. Test sets were scored only after selection. Every hyperparameter is saved in the checkpoint and in the `run_start` log record, and resume is tested.

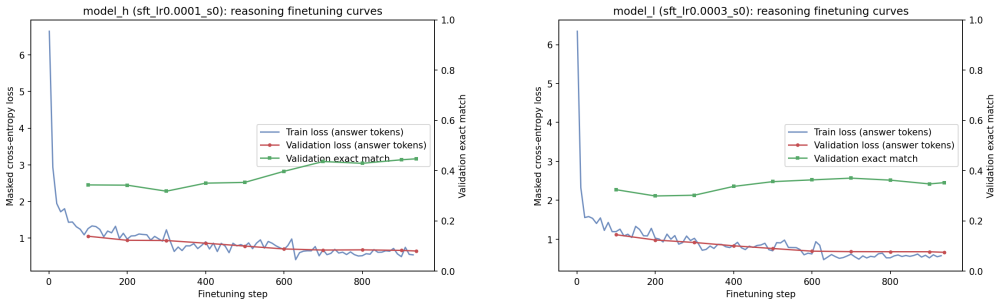


Figure 5: Finetuning curves (seed 0): masked training and validation loss, and validation EM. The loss keeps falling after EM has flattened.

3.3 Results: pretrained vs finetuned

Table 12: Exact match (mean ± std over 3 seeds). Cand. = candidate-choice accuracy on `test_both`: candidates ranked by log-likelihood, with no generation.

Model	Condition	val	test_id	test_entity	test_wording	test_both	Cand.
H	Pretrained, 0-shot	0.000	0.000	0.000	0.000	0.000	0.221
	Pretrained, 3-shot	0.001	0.000	0.000	0.000	0.000	0.249
	SFT	0.408 ± 0.038	0.388 ± 0.049	0.136 ± 0.043	0.331 ± 0.056	0.102 ± 0.020	0.319
	SFT + replay	**0.434** ± 0.029	**0.422** ± 0.045	0.129 ± 0.009	**0.397** ± 0.036	0.102 ± 0.005	0.339
L	Pretrained, 0-shot	0.000	0.000	0.000	0.000	0.000	0.265
	Pretrained, 3-shot	0.000	0.000	0.000	0.000	0.000	0.267
	SFT	0.368 ± 0.003	0.353 ± 0.007	0.103 ± 0.002	0.322 ± 0.002	0.085 ± 0.003	0.273
	SFT + replay	**0.371** ± 0.005	**0.368** ± 0.011	**0.134** ± 0.004	**0.352** ± 0.011	**0.107** ± 0.006	0.292

- **Pretrained models score 0.000 EM**, zero-shot and 3-shot. They almost never stop after a name (1–2% zero-shot), and candidate choice is below chance (0.22–0.27), so there is no hidden skill behind the format errors.

- After SFT the format is learned fully: all finetuned models stop 100% of the time. On seen names, candidate choice closely matches EM (H SFT: 0.386 vs 0.388).
- **The comparison does not transfer to new names.** On `test_both`, candidate choice is 0.27–0.34 for every finetuned model, around chance (0.30). Held-out wording costs little (H 0.422→0.397, L 0.368→0.352). Unseen names cost a lot (∼0.10).

Table 13: EM by task type (SFT + replay, 3-seed mean) and error causes on `test_both` (counts per seed, out of 1,000).

	cmp2	cmp3	ext3	pair3	ext4	Error cause	H pre.	H repl.	L pre.	L repl.
H test_id	0.503	0.375	0.411	0.508	0.308	no stop	139	0	20	0
H test_both	0.219	0.264	0.012	0.007	0.003	reversed extreme	150	3	16	21
L test_id	0.365	0.276	0.414	0.529	0.254	middle entity	354	0	29	17
L test_both	0.190	0.251	0.030	0.032	0.029	equal-case miss	38	267	5	348
Task chance	0.333	0.250	0.333	0.333	0.250	out-of-prompt name	21	**598**	2	**501**
						format failure	298	30	928	5

- **With unseen names the chain tasks break, but the value tasks do not.** `cmp2`/`cmp3` stay at 0.19–0.26, while `ext3`/`pair3`/`ext4` drop to 0.00–0.03. In a `cmp` prompt each name sits next to its own number; in a chain task names only appear in relation to each other.
- **On seen names the main error is the wrong direction** (reversed extreme, about 350 per seed; no out-of-prompt names at all). **On unseen names 56–67% of errors are a training name that is not in the prompt.** The model ties answers to names it has memorised. One example: for unseen *Nabin* the model copies the first token and then completes it as the training name *Navinā*.
- Attribute and wording family do not matter much. Attributes on `test_id` are within 0.09 of each other. The held-out family F4 (H 0.397, L 0.352) lies inside the F1–F3 range (H 0.392–0.442, L 0.350–0.391).
- Examples (replay, seed 0). Correct, unseen names and F4: *Indu ke mukābale Roshanī chhoṭī hai. Ashok ke mukābale Indu chhoṭī hai. Inmeñ sabse lambā kaun hai?* gives *Ashok*. Wrong (middle entity): *Abhishek, Shālinī se bhārī hai. Shālinī, Kīrti se bhārī hai. Sabse halkā kaun hai?* gives *Shālinī* (gold *Kīrti*). Nepali, wrong (out-of-prompt name): *Ajitle 28 varsha pūrā gareko cha. Sumitrāle 28 varsha pūrā garekī chin. … jeṭho ko ho?* gives *Sudip* (gold *barābar*).

Table 14: Forgetting (Phase-2 validation perplexity) and effect of training-set size (EM; 1k/5k use seed 0, plain SFT).

Val perplexity	Model H	Model L	Train examples	1,000	5,000	20,000
Pretrained	30.34	51.23	H test_id / test_entity	0.296 / **0.227**	0.330 / 0.158	0.388 / 0.136
SFT	142.4 ± 85.2	418,137 ± 313,846	L test_id / test_entity	0.280 / **0.154**	0.351 / 0.108	0.353 / 0.103
SFT + replay	**35.51** ± 0.11	**76.32** ± 0.19				

- **Plain SFT damages the language model.** Model L at its higher LR ends up with perplexity above 400k. Selecting on EM alone preferred this LR by 1.4 points.
- Replay prevents this without hurting the task: EM is equal or better on every split except H `test_entity` (0.136 to 0.129), and the seed variance shrinks. SFT + replay is the recommended model.
- More data increases memorisation: seen-name accuracy rises and unseen-name accuracy falls. The extra examples reuse the same 140 names.

3.4 Attention after finetuning

The Phase-2 toolkit, unchanged, was run on the pretrained model and on the replay seed-0 model. The plain-SFT Model L was not used, because its forgetting would mix into the changes. Inputs were 200 reasoning prompts and the same 8×192-token validation text as before. **Pointer score**: at the *uttar:* position, attention on the correct entity's tokens minus the mean attention on each distractor's tokens.

Table 15: Pointer score and mean attention distance on held-out text, pretrained → finetuned.

Pointer score	Model H	Model L	Distance (text)	L0	L5	**L6**	L8	L9
Best head, pretrained	0.001	0.000	Model H	43.5→44.4	29.5→30.5	17.5→16.6	24.4→23.2	34.6→32.9
Best head, finetuned	L9H5 0.091	L9H0 0.101	Model L	41.1→43.8	32.0→36.1	17.4→18.1	36.8→34.0	30.2→31.7
Largest gain, layers 0–6	+0.008	+0.035						
Layer-9 mean	−0.026→+0.057	−0.018→+0.037						

- **Finetuning created answer-pointer heads at the top of the network.** Before finetuning no head prefers the answer; afterwards all six layer-9 heads of Model H improve (+0.03 to +0.12), and Model L's gains are in L9H0 and L9H2. Layer 0 heatmaps look the same before and after.
- Local vs long-range attention on ordinary text did not change: every layer's distance stays within 4.1 tokens

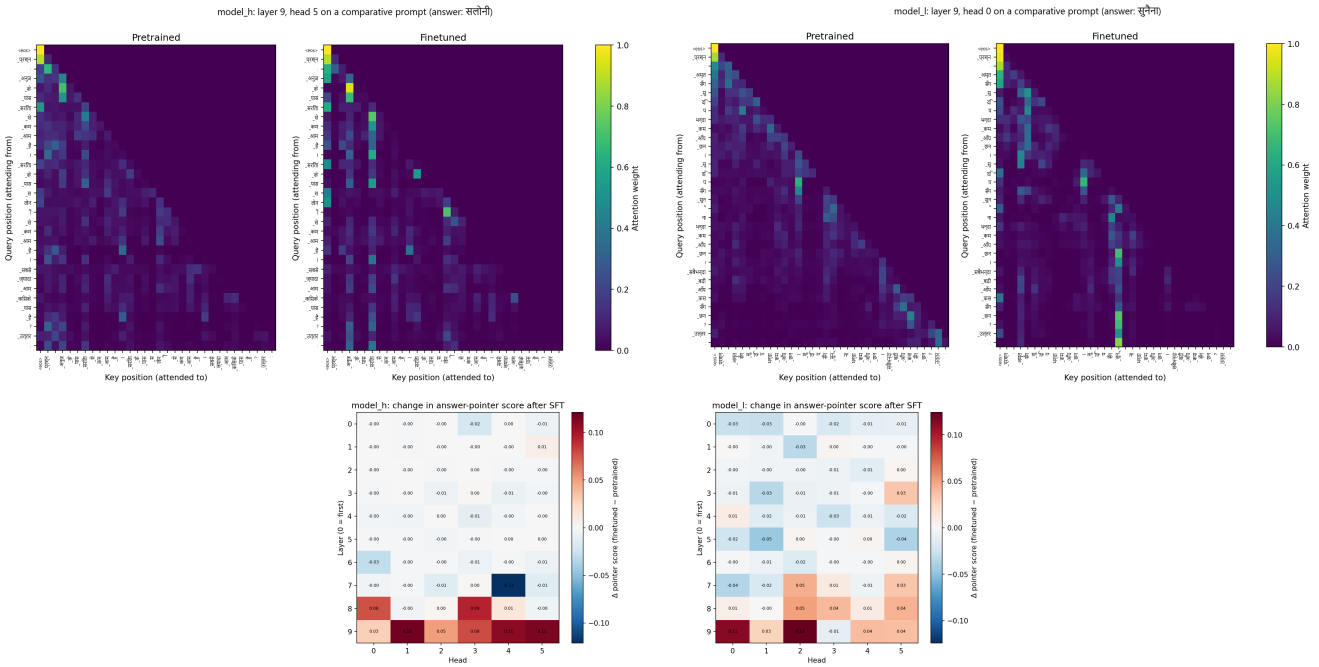


Figure 6: Top: pretrained vs finetuned layer-9 heads on an `ext3` prompt (H L9H5, L L9H0). After finetuning the heads form vertical stripes on entity tokens. Bottom: change in pointer score per layer and head (H left, L right); the gains are in layers 8–9.

and normalised entropy within 0.04. The layer-6 focusing layer survives in both models.

- One change is specific to the task: on reasoning prompts, Model L's layers 7–9 become sharper (normalised entropy 0.72/0.61/0.69 → 0.55/0.54/0.59) while the same layers on text barely move. Model H's change is in which tokens the heads attend to, not in how spread out the attention is.
- Head roles are unchanged on text. On reasoning prompts, H loses its previous-token head and L loses its 3 sink heads (diffuse 37→35, mixed 19→24). A pointer score of about 0.1 is a weak signal, which is consistent with the model falling back to memorised names when the names are new.

4 Answers to the Four Questions

1. How did data scale and quality differ?

- Scale: **Nepali ended up larger** (572.5M vs 517.6M tokens, 5.53M vs 3.58M documents), because the crawl made up for Nepali having less material online. Model L saw 24.0 tokens per parameter, Model H 21.6.
- Quality: **Nepali was noisier.** About 85% of its manual text is fast-crawled news (against Hindi's much larger Wikipedia share), it needed lower LID and quality thresholds, and it had 43% held-out collisions from syndicated sentences.
- The Phase-3 data was matched on purpose: same design, sizes, seeds and leakage checks, with each language's own names, grammar and digits.

2. How do the LM and reasoning results compare across the two tiers?

- Language modelling: H is better per token (PPL 31.01 vs 52.16), **L is better per byte** (0.4561 vs 0.4819).
- Reasoning: **H is about 5 points better on seen names** (0.422 vs 0.368; held-out wording 0.397 vs 0.352). On unseen names they are equal (0.129 vs 0.134; 0.102 vs 0.107).
- Robustness: L needed a 3× higher LR and forgot far more without replay (PPL 418k vs 142). Attention is organised the same way in both (layer 6, about 17.4 tokens).

3. Which tokenizer and corpus factors affected the lower-resource model most?

- Token granularity: 12.51 vs 10.28 bytes/token. This inflates Model L's perplexity, and more names are split: only 58% of Nepali names are one token, against 85% for Hindi (mean 1.54 vs 1.18 tokens).
- Attached postpositions (*-ko, -le, -bhandā, -sañga*) raise fertility (1.380 vs 1.273), and in the prompts the bare answer name never appears as a separate word.
- Per-language normalisation: chandrabindu, the final virama and Devanagari digits. A pipeline tuned on Hindi would have fed Model L spellings it never saw.
- Composition matters more than size: the noisier crawl shows up in generation (stray Latin characters, lower unique-word ratio) rather than in BPB.

4. What evidence explains the differences?

- Perplexity gap = tokenizer: the bits/token ratio (1.151) is smaller than the bytes/token ratio (1.216), and the BPB ranking reverses.
- Model L's better BPB is consistent with 11% more training tokens.
- Model H's reasoning lead is about copying names: more of its names are single tokens, Model L's name-completion errors (*Nabin*→*Navinā*), and the gap disappears once memorisation dominates on unseen names.
- SFT teaches format rather than reasoning: stop rate 1–2%→100%, candidate choice on `test_both` near chance, 56–67% out-of-prompt names, and pointer gains only in layers 8–9.

5 Bonus: Model H Without Positional Encoding (NoPE)

- Setup: Model H's only position signal is RoPE on Q and K. A flag `use_rope=False` removes it. Everything else is the same as the standard run: 23,846,784 parameters, data, tokenizer, optimizer, schedule, seed 42 and 3,936 steps on Kaggle 2×T4. The causal mask is kept. A test checks that identical tokens get identical layer-0 attention when RoPE is off.
- Evaluation: the full Phase-2 suite with the unchanged scripts, plus two position probes (loss by position; continuation loss after an intact vs a shuffled context). Code, results and figures are on the `bonus` branch (`hindi/nope/`, `report/bonusPhaseReport.md`).

Table 16: Standard Model H vs NoPE Model H (test split unless noted).

	Standard	NoPE		Standard	NoPE
Final val loss	3.4237	3.5681	Greedy chrF / BLEU / ROUGE-L	16.60 / 1.47 / 11.82	15.10 / 0.87 / 10.18
Test PPL / BPB	31.01 / 0.4819	35.72 / 0.5017	T=0.5 chrF / BLEU / ROUGE-L	21.34 / 1.86 / 13.12	20.39 / 1.47 / 12.19
Loss gap by position (256–1023)	–	+0.145 nats	T=1.0 chrF	22.96	22.85
Shuffle cost (order sensitivity)	+0.264	+0.229 (87%)	Greedy Dist-1 / Dist-2 / Rep-4	0.062 / 0.152 / 0.732	0.049 / 0.115 / 0.790
Throughput	75.7k tok/s	89.8k tok/s	Norm. entropy L0–4 / L6–8	0.84–0.90 / 0.65–0.67	0.89–0.96 / 0.55–0.58
Head roles (diff/mixed/long/prev)	46/9/4/1	42/12/5/1	Distance L3 / L6 / L9	33.6 / 17.5 / 34.6	51.6 / 8.7 / 47.8
Top prev-token head	L8H1 0.554	L8H0 0.726	Prev-token mass L0–4	0.03–0.09	0.02–0.03

What breaks without position information

- Early layers cannot attend by position. With content-only Q/K, the previous token looks like any other similar token. Layers 0–4 turn into near-uniform averaging layers. Uniform attention over $i+1$ visible tokens gives each a weight of $1/(i+1)$, which later layers can use as a rough position signal.
- Local attention still appears, but deeper: layers 6–8 become more local than in the standard model (8.7 tokens at layer 6) and develop a stronger previous-token head. Fewer layers remain above them to use it.
- The cost is roughly constant: the loss gap appears within the first 16 positions (peak +0.18) and stays flat at +0.145 nats from position 256 to 1023. Perplexity rises ×1.15 and BPB +4.1%. During training the gap stops shrinking after step 3,000.
- Loops are harder to escape: identical tokens have identical keys, so a head cannot pick out "the copy I just wrote". Greedy repetition rises (0.732→0.790). In one sample NoPE looped for 115 words, while the standard model's longest loop was 18. At $T \geq 1$ the two models are the same.
- What still works: word order is still used (87% of the standard model's shuffle cost), long contexts are not hurt more than short ones, and training stays stable.

6 Independence and Reproducibility

- `scripts/verify_independence.py`: no cross-language paths; document overlap 0; tokenizer files differ (5,695 of 16,000 pieces shared); the pretrained checkpoints share no storage and no identical tensors (embedding cosine +0.0045); all 20 finetuned checkpoints point back to their own language (cosine ≥0.875 to their own pretrained model, ≤0.0046 to the other).
- Tests: 496 pass (pytest). Examples: attention path equivalence, the causality test, rank-disjoint sampling, crash during save, append-only logs, attention statistics against closed forms, solver agreement, splits free of leakage, and a byte-identical dataset rebuild.
- Commands (full steps and all Drive links for datasets, pretrained and finetuned checkpoints are in `README.md`):
  Phase 1: `scripts/build_corpus.py`, `scripts/tokenizer_sweep.py`
  Phase 2: `torchrun -nproc_per_node=2 <lang>/train/pretrain.py`, `<lang>/eval/evaluate.py`, `<lang>/eval/attention_report.py`
  Phase 3: `scripts/run_phase3.py -lang <lang>`, `scripts/phase3_compare.py`, `scripts/verify_independence.py`